Тема 14

• type-casting-преобразуване от един тип в друг

int x = -1 $\boxed{11\ldots111}$

unsigned y = x // 2^{32-1}

Base ← Der Der* d = new Der();

Base* = d (=(Base*)d)

↳ c-style cast

Видове:

• static_cast

→ не прави runtime check

→ ако преобразуването е неуспешно → CRASH

→ ползва се само когато типът, в който преобразуване, със сигурност е коректен

Base* bPtr = static_cast<Base>(ptr) → Der*

! upcasting Der* → Base* (губи се конкретика)

• downcasting Base* → Der* (типът трябва да се провери по някакъв начин)

• dynamic_cast

→ прави runtime check

→ при неуспешно преобразуване връща nullptr (ако работим с указатели)

→ по-бавно от static_cast

→ ползва се предимно за downcasting (няма смисъл за upcasting)

Base

A → Base ← B

```
f(Base* ptr)
    if(A* aPtr = dynamic_cast<A>(ptr)) { //логика за A }
    else if(B* bPtr = dynamic_cast<B>(ptr) { //логика за B }
```

→ при грешен опит за dynamic_cast на референция се хвърля std::bad_cast (тоне не нямам неутрална стойност)

→ runtime проверката се извършва чрез виртуална таблица ⇒ можем да downcast-ваме само ако имаме поне 1 виртуална функция

• const_cast (НЕ ползваме)

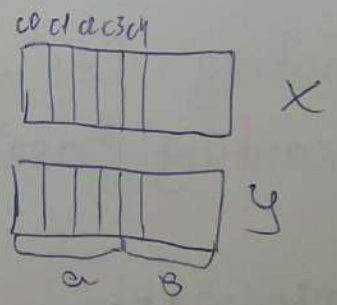
→ манипулиране на константни обекти, с които работим през указатели и референции

→ за да премахнем const, трябва да сме сигурни, че обектът не е бил константен при създаването си (иначе undefined behaviour)

• reinterpret_cast

→ реинтерпретация на памет

```
class X { char c[5]; };
class Y { int a,b; };
```



Писане в двоичен файл:

```
f.write(reinterpret_cast<const char*>(&var), size);
```

• C-style cast

→ последователност от cast-ове

1. const_cast
2. static_cast
3. static_cast + const_cast
4. reinterpret_cast
5. reinterpret_cast + const_cast

```
A* aPtr=(A*)bPtr;
```

design patterns – обобщени добри практики за решаване на често възникващи проблеми

→ creational patterns – осигуряват създаването на обекти, като скриват логиката по създаването

→ structural patterns – начин за създаване на по-сложни обекти, използвайки инструменти като наследяване и композиция

→ behavioural patterns – осигуряват комуникацията между обекти

→ singleton (creational) – осигурява само една инстанция на клас, до който има глобален достъп

⊕ – само 1 инстанция
– глобален достъп
– lazy initialization

⊖ – обвързваме се с конкретна инстанция
– не може да се тества при използване с друг код

→ factory (creational) – статична функция (може в клас), която при подадени аргументи връща инстанция

→ factory method – интерфейс с точно 1 create метод; всеки наследник представя имплементация на create, в която решава какъв обект да върне

→ abstract factory – интерфейс с 1 create метод за всеки обект, който създава (обектите са свързани логически)

⊕ – генерално място за създаване на обекти
– по всяко време можем да добавим наследник на BaseFactory

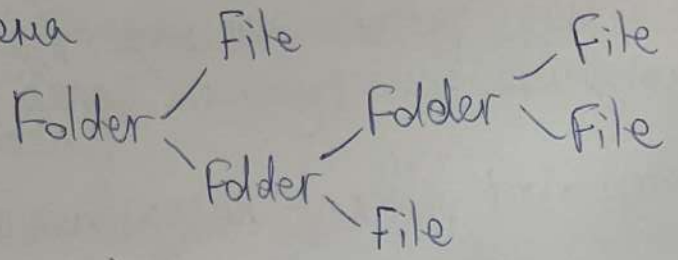
⊖ – добавя ново ниво на абстракция и така усложнява кода
– грижа за живота на Factory

→ prototype (creational) - създаване на копие на обект без интересуване от типа му (clone)

→ composite pattern (structural) - композиране на обекти в дървовидна структура (листа, междинни върхове)

пример: файлова система

File, Folder(File**)



→ flyweight pattern (structural) - събира повече обекти в паметта, споделяйки общите им ресурси; подобрява бързодействието, ако създаването е тежка операция

→ реализира се с хеш/дърво, обектите са immutable (пример: StringPool)

→ iterator (behavioural) - начин за работа с колекции без да се интересуваме каква е тя

→ итератор - указател към конкретния елемент

*it - връща елемента ; op== / !=

it++ / it-- → отиди на следващия / предишния

→ command pattern (behavioural) → при програма, която получава заявки

Command
virtual exec() = 0;

↑ createCmd exec() override

↑ printCmd exec() override

→ командите се обвиват в обекти

→ може да се забави изпълнението им

→ възможно е undo, ако се пазят в някакъв stack

→ командите могат да се използват като параметри

SOLID principles – 5 принципа за добър код

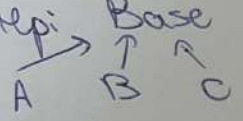
S – single responsibility – един компонент трябва да има точно 1 отговорност (висок cohesion)

Пример: Person не трябва да знае как се контролира char*
⇓
string

readAndWrite() → read()
write()

→ O – open/close – open for extension/closed for modification

Пример:



f(Base* ptr) – не трябва да променя логиката си при добавяне на наследник

→ L – Liskov's substitution – трябва да използваме указателя към базовия клас без да се интересуваме от конкретния тип (dynamic_cast го нарушава !)

Пример:

```
class Car
  getCabinSize()

class RaceCar: Car {
  getCabinSize() override
    throw "Not implemented";
  getCockpitSize()

f(Car& c)
  c.getCabinSize(); //throw-ва за raceCar
```

→ I – interface segregation – потребителят не е длъжен да имплементира интерфейс, който не трябва да използва

```
class OfficeMachine
  virtual print() = 0
  virtual fax() = 0
  virtual scan() = 0

class Printer: OfficeMachine
  print {...}
  scan {...}
  fax {throw}

class Fax: OfficeMachine {
  print {throw}
  scan {throw}
  fax {...}
```

решение:
IPrint { print() = 0; } ; IScan { scan() = 0; } ; IFax { fax() =
Printer : IPrint, IScan {..}; Fax: IFax {..};

→ D - dependency inversion - модулите на високо ниво ~~ИЕ~~ трябва да зависят от модулите на ниско ниво
с директен достъп до данните

Пример: ProductCatalog
ProductTypeIProvider p;
↳ възниква проблем при нужда от provider на друг тип

Решение: ProductCatalog
Provider* pPtr; //агрегация и се възползваме
// от абстракциите

Заемаме външни зависимости вместо да създаваме вътрешно

• Пример за Singleton+Factory

```
#define st size_t
class Base {
 public:
   virtual ~Base() = default;
};
class A: public Base { };
class B: public Base { };
class SingletonFactory {
   st current = 0;
   st count = 0;
   st* numbers = nullptr;
   ~SingletonFactory() {
       delete[] numbers;
   }
```